

ME315_Lab05_Elisa

Laboratório 5

Criar uma função que:

- Receba um valor de TAIL_NUMBER (por exemplo, N431WN);
- Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip);
- Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima);
- O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

Resolução por partes

Metodologia: a ideia para a construção desse algoritmo foi “separar” a tarefas em pequenos passos, ou seja, funções que, ao final, quando reunidas na função principal (main -) trará as informações pedidas e atenderá o anunciado.

Carregando os pacotes

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1    ✓ readr      2.1.5
✓ forcats    1.0.1    ✓ stringr    1.5.1
✓ ggplot2    3.5.1    ✓ tibble     3.2.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr      1.2.2
— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(leaflet)
library(ggcal)
```

Configurando o caminho dos arquivos

```
caminho_flights <- file.path("dados", "flights.csv")
caminho_airports <- file.path("dados", "airports.csv")

file.exists(caminho_flights) # Confirmando se o arquivo existe
```

[1] TRUE

```
file.exists(caminho_airports) # Confirmando se o arquivo existe
```

[1] TRUE

Primeiro, vamos carregar a tabela de aeroportos (por ser pequena, conseguimos ler inteira sem comprometer a memória da máquina que vai executar) e renomear algumas colunas com nomes mais intuitivos.

```
# Carrega a tabela de aeroportos e renomeia as colunas que interessam para nomes mais claros.
carrega_aeroportos <- function(arquivo_aeroportos) {
  readr::read_csv(arquivo_aeroportos, show_col_types = FALSE) %>%
    dplyr::select(
      codigo_aeroporto = IATA_CODE,
      nome_aeroporto   = AIRPORT,
      latitude_aeroporto = LATITUDE,
      longitude_aeroporto = LONGITUDE
    )
}
```

Em seguida vamos criar uma função que vai ler o arquivo flights.csv, em razão do tamanho do arquivo, vamos carregar em chunks, além disso, adicionei uma condição que confere se a aeronave de fato fez algum voo.

```
# Lê flights.csv EM PARTES (chunks de 100 mil linhas), mantendo em memória apenas os voos da aeronave
le_voos_da_aeronave <- function(arquivo, tail_number) {

  filtra_pedaco_pela_aeronave <- function(pedaco_de_dados, posicao_no_arquivo) {
    dplyr::filter(pedaco_de_dados, TAIL_NUMBER == tail_number)
  }

  voos_da_aeronave <- readr::read_csv_chunked(
    file          = arquivo,
    callback      = readr::DataFrameCallback$new(filtra_pedaco_pela_aeronave),
    chunk_size    = 100000,
    show_col_types = FALSE
  ) # armazenamento dos voos fracionados

  if (nrow(voos_da_aeronave) == 0) {
    stop("Nenhum voo encontrado para a aeronave '", tail_number, "'.")
  } # Condição de que a aeronave tenha feito voos

  voos_da_aeronave
}
```

Nosso próximo passo será organizar cronologicamente os voos e construir as velocidades médias

```
# Monta a data completa do voo, ordena por data/hora de partida, numera cada trecho na ordem cronológica
organiza_trajeto_cronologico <- function(voos_da_aeronave) {
  voos_da_aeronave %>%
```

```
dplyr::mutate(
  data_do_voo = lubridate::make_date(YEAR, MONTH, DAY)
) %>%
dplyr::arrange(data_do_voo, SCHEDULED_DEPARTURE) %>%
dplyr::mutate(
  numero_do_trecho = dplyr::row_number(),
  velocidade_media_mph = DISTANCE / (AIR_TIME / 60)
)
}
```

Aqui, juntamos as coordenadas tanto da origem quanto do destino, importante dizer também que esse código através do comando "rename" trouxemos nomes mais intuitivos para as variáveis facilitando tanto a validação do código em si quanto a interpretação de quem lê.

Outro comando é o left join usado para concatenar as informações das tabelas.

```
adiciona_coordenadas <- function(voos_ordenados, tabela_aeroportos) {
  voos_ordenados %>%
    # Juntando os dados necessarios
    dplyr::left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "codigo_aeroporto")) %>%
    dplyr::rename(
      nome_origem = nome_aeroporto,
      latitude_origem = latitude_aeroporto,
      longitude_origem = longitude_aeroporto
    ) %>%
    dplyr::left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "codigo_aeroporto")) %>%
    dplyr::rename(
      nome_destino = nome_aeroporto,
      latitude_destino = latitude_aeroporto,
      longitude_destino = longitude_aeroporto
    ) %>%
    dplyr::filter(!is.na(latitude_origem), !is.na(latitude_destino))
}
```

Aqui estamos olhando para o mapa do trajeto utilizando o pacote leaflet

```
constroi_mapa_trajeto <- function(trajeto_da_aeronave) {

  faixa_de_velocidades <- range(trajeto_da_aeronave$velocidade_media_mph, na.rm = TRUE)

  mapa_trajeto <- leaflet::leaflet(trajeto_da_aeronave) %>%
    leaflet::addProviderTiles(leaflet::providers$CartoDB.Positron)

  # Uma linha por trecho, na ordem cronológica em que a aeronave voou; espessura proporcional
  for (indice_do_trecho in seq_len(nrow(trajeto_da_aeronave))) {

    trecho_atual <- trajeto_da_aeronave[indice_do_trecho, ]

    espessura_da_linha <- scales::rescale(
      trecho_atual$velocidade_media_mph,
      to = c(1, 8),
```

```

    from = faixa_de_velocidades
  )

texto_do_popup <- paste0(
  "<b>Trecho ", trecho_atual$numero_do_trecho, "</b><br>",
  trecho_atual$ORIGIN_AIRPORT, " &rarr; ", trecho_atual$DESTINATION_AIRPORT, "<br>",
  "Data: ", trecho_atual$data_do_voo, "<br>",
  "Velocidade média: ", round(trecho_atual$velocidade_media_mph, 0), " mph"
)

mapa_trajeto <- mapa_trajeto %>%
  leaflet::addPolylines(
    lng      = c(trecho_atual$longitude_origem, trecho_atual$longitude_destino),
    lat      = c(trecho_atual$latitude_origem, trecho_atual$latitude_destino),
    weight    = espessura_da_linha,
    color     = "#2c7fb8",
    opacity   = 0.8,
    popup     = texto_do_popup
  )
}

# Marca cada aeroporto único visitado
aeroportos_visitados <- dplyr::bind_rows(
  trajeto_da_aeronave %>%
    dplyr::select(codigo = ORIGIN_AIRPORT, nome = nome_origem,
                  latitude = latitude_origem, longitude = longitude_origem),
  trajeto_da_aeronave %>%
    dplyr::select(codigo = DESTINATION_AIRPORT, nome = nome_destino,
                  latitude = latitude_destino, longitude = longitude_destino)
) %>%
  dplyr::distinct(codigo, .keep_all = TRUE)

mapa_trajeto %>%
  leaflet::addCircleMarkers(
    data      = aeroportos_visitados,
    lng       = ~longitude,
    lat       = ~latitude,
    radius     = 5,
    color      = "#d95f0e",
    fillOpacity = 1,
    popup      = ~paste0("<b>", codigo, "</b><br>", nome)
  )
}

```

Função principal -> analisa_aeronave

Nossa função main que absorve nossos resultados anteriores. Aqui também nomeamos as variáveis de forma clara e objetiva.

```

analisa_aeronave <- function(tail_number,
                             arquivo,
                             arquivo_aeroportos = file.path("dados", "airports.csv")) {

```

```
tabela_aeroportos <- carrega_aeroportos(arquivo_aeroportos)
voos_da_aeronave <- le_voos_da_aeronave(arquivo, tail_number)
voos_ordenados <- organiza_trajeto_cronologico(voos_da_aeronave)
trajeto_da_aeronave <- adiciona_coordenadas(voos_ordenados, tabela_aeroportos)
mapa_trajeto <- constroi_mapa_trajeto(trajeto_da_aeronave)

list(
  tabela = trajeto_da_aeronave,
  grafico = mapa_trajeto
)
```

Teste

Aqui temos um exemplo de uso, onde apresentamos tanto a tabela com diversas informações quanto o mapa que constroi todas as rodas de voos feitos pela aeronave.

```
resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("dados", "flights.csv"),
  arquivo_aeroportos = file.path("dados", "airports.csv")
)

resultado$tabela
```

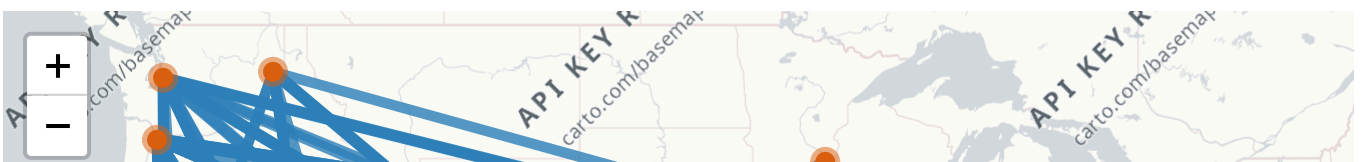
A tibble: 1,780 × 40

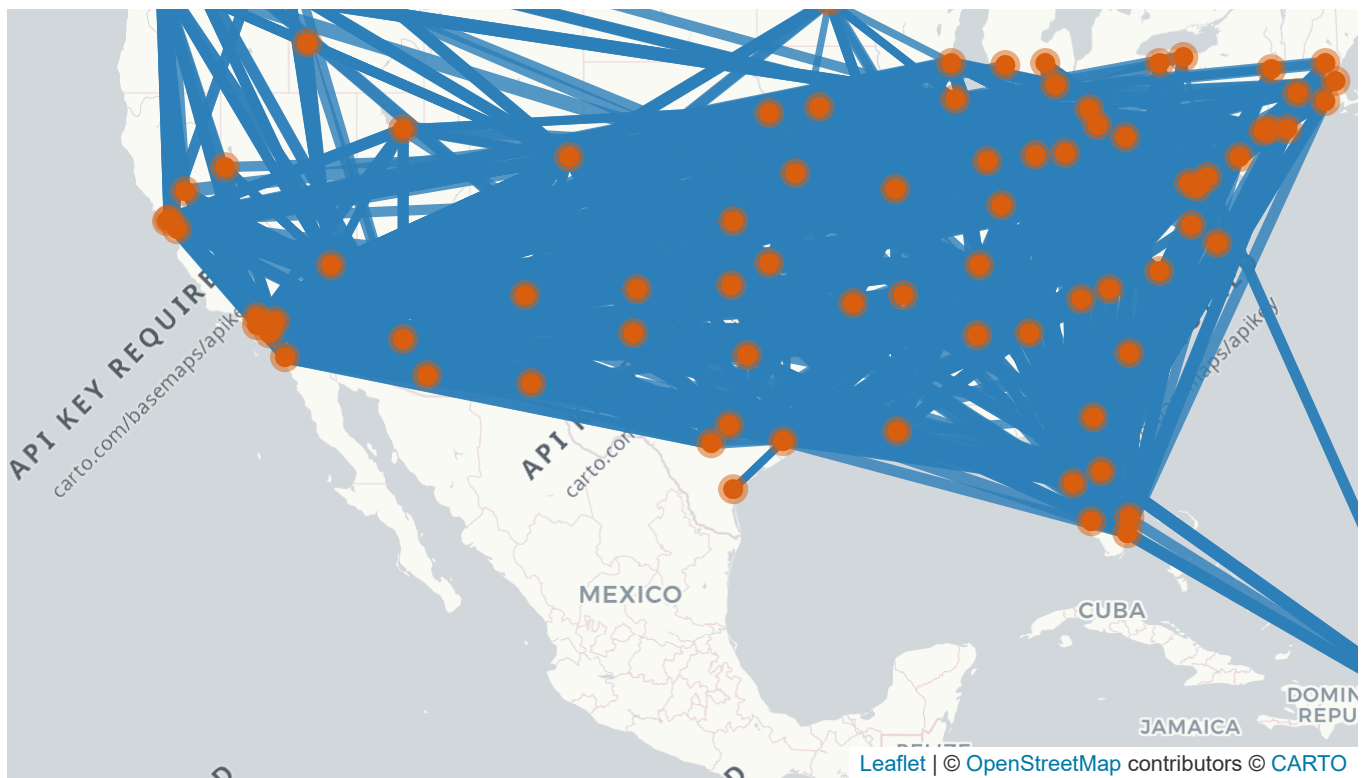
	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>
1	2015	1	1	4	WN	1762	N431WN
2	2015	1	1	4	WN	3564	N431WN
3	2015	1	1	4	WN	3564	N431WN
4	2015	1	1	4	WN	3564	N431WN
5	2015	1	2	5	WN	1793	N431WN
6	2015	1	2	5	WN	1793	N431WN
7	2015	1	2	5	WN	1793	N431WN
8	2015	1	2	5	WN	1793	N431WN
9	2015	1	2	5	WN	1793	N431WN
10	2015	1	2	5	WN	3144	N431WN

i 1,770 more rows

i 33 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
 # SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
 # TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
 # AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
 # SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
 # DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```
resultado$grafico
```





Código com horario de compilacao

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 21:30:03 -03"
```